

Vanier College
Computer Science Technology 420.B0
System Development
420-436-VA

System Development Project

Orange Team

04/13/2026
Fairoz Aly, Karim Fahd, Daniel Perrotta, Cyril Efren Fabro
Le Châteaubriand

Table of contents

1. Executive Overview.....	2
2. Business problem.....	3
3. Narrative Description.....	4
4. Appendix 1 - Prototype Interface.....	5
5. Appendix 2 - Data Dictionary.....	7
6. Appendix 3 - Query descriptions.....	14
7. Team Logbook.....	15

1. Executive Overview

Building on the system design established in the previous deliverables, this document focuses on the database design that will support the online booking and client management system for Le Châteaubriand. Having already identified the business problem, gathered client requirements, and produced the necessary UML diagrams, the team is now translating that into a concrete relational database structure. This deliverable outlines the physical and logical design of the database, including an ER diagram, a data dictionary, and an analysis of query optimization. The goal is to ensure that the system can store, retrieve, and manage data reliably and efficiently, while remaining simple enough for the owners, Tony and Pat, to benefit from without requiring technical expertise on their end.

2. Business problem

The inefficiency of the present booking procedure, which is handled exclusively in person, remains the core problem our system aims to resolve. Collecting essential information such as dates, guest counts, menu preferences, bar service, and budget currently requires multiple in-person interactions, leading to wasted time, duplicated effort, and an increasing administrative burden as the business grows. Furthermore, client records are currently maintained through physical contracts and Excel spreadsheets, with no centralized system for tracking or managing that information. The database designed in this deliverable directly addresses that gap by providing a structured, centralized store for all client and event data, enabling the owners to manage bookings, track event statuses, and access client information from a single system.

3. Narrative Description

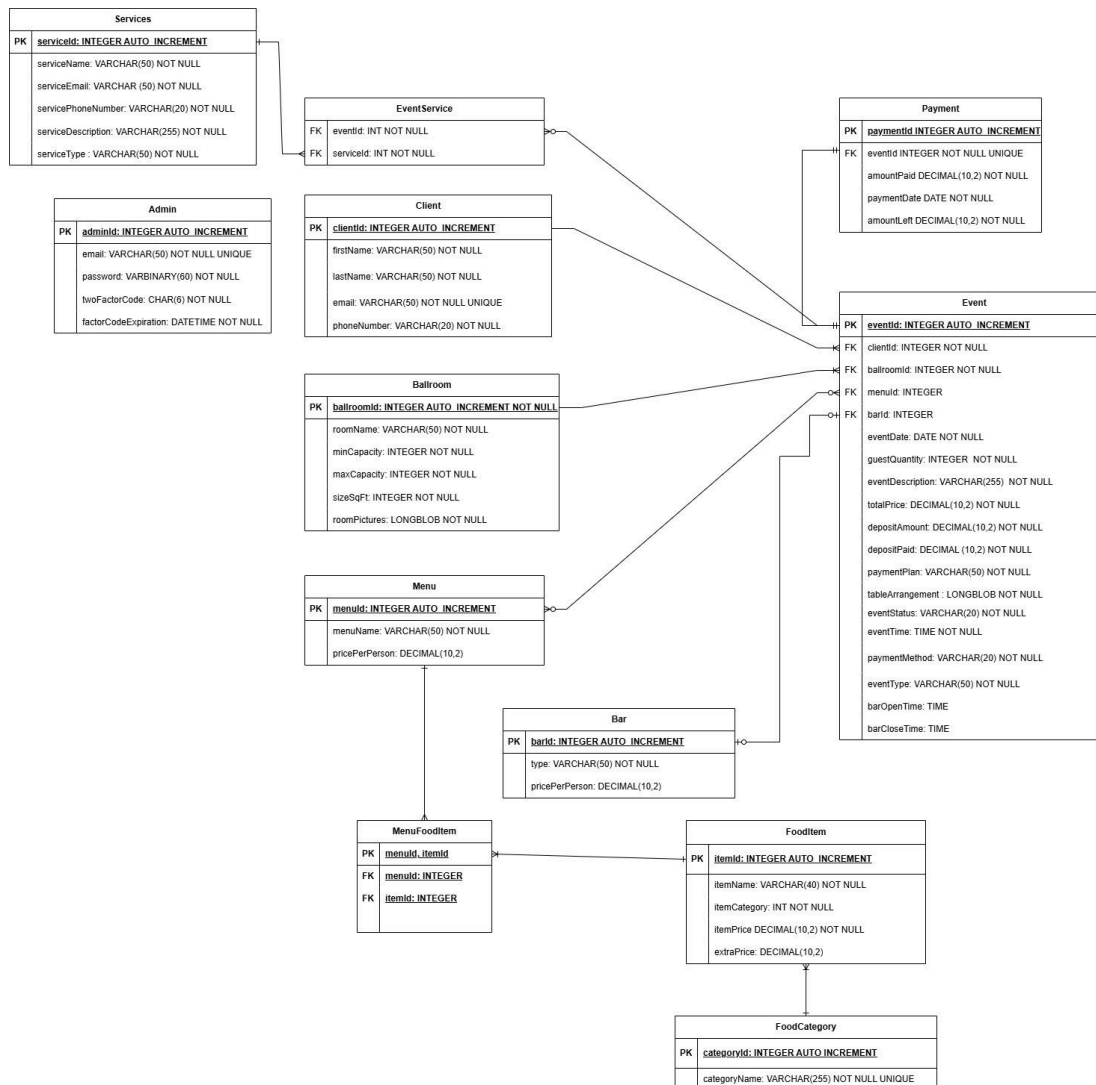
There are two actors in this system: the owner and the client. Each interacts with the database in distinct ways that reflect their role in the booking process.

When a client visits the website, they can fill out an online booking form that captures their personal information, including their name, email, and phone number, which is stored in the client table. As part of the form, the client selects an available date and time, chooses a ballroom based on their guest count, picks a menu package or catering option from the available food items, and optionally requests bar service. All of these choices are brought together and stored as a single record in the event table, which acts as the central entity of the database. If the client wishes to add third-party services such as entertainment or decoration, those are linked to their event through the eventService junction table. Once the form is submitted, the event is saved with a status of Pending and a confirmation is sent to the client.

When the owner logs into the admin dashboard using their credentials, which are securely stored in the owner table with a hashed password and two-factor authentication code, they gain access to all client and event records. The owner can view the full list of clients, search and filter by name or event details, and click into any individual client to see their complete event information, including the selected ballroom, menu, bar preference, guest count, payment status, and third-party services requested. The owner can also edit or delete client records as needed, and view all upcoming events on a calendar. As events are completed or cancelled, the owner updates their status accordingly, keeping the system current and reducing the administrative overhead that previously came with managing everything manually.

The database uses MySQL and consists of eleven tables: owner, client, event, ballroom, menu, foodItem, foodCategory, menuFoodItem, bar, services, eventService, and payment. This structure ensures that data is stored without unnecessary repetition, relationships between entities are enforced through foreign keys, and the system can scale comfortably as the business continues to grow.

4. Appendix 1 - Prototype Interface



Comparison

The ER diagram and class diagram share the same core entities and overall structure, but differ in several important ways to show the transition from object-oriented design to relational database design.

Similarities

The main entities are consistent across both diagrams. Client, Event, Ballroom, Bar, Menu, FoodItem, Admin, and the third-party services concept all appear in both. The central role of the Event entity, connected to Client, Ballroom, Bar, and Menu, is preserved in the ER diagram exactly as designed in the class diagram. The Client and Admin attributes, such as name, email, phone number, password, and two-factor authentication fields, are also consistent between both.

Differences

The most significant difference is how inheritance was handled. The class diagram uses inheritance extensively. Bar has PaidBar, OpenBar, PremiumBar subclasses, Ballroom has Grand, Princess, and Royal subclasses, and Menu has MainMenu, MidnightTable, and buffet subclasses. Relational databases don't support inheritance natively, so these were reduced to single tables with type fields instead. For example, Bar became a single table with a 'barType' field using a CHECK constraint to distinguish between types.

Similarly, ThirdPartyServices and its subclasses Decor, DJ, and Catering were consolidated into a single 'services' table with a 'serviceType' field rather than separate tables.

The class diagram included methods such as calculateTotalPrice(), reserveRoom(), and generateInvoice(), and these are behaviours that belong to the application layer and have no place in the database design, so they were dropped from the ER diagram entirely.

Finally, junction tables 'menuFoodItem' and 'eventService' were added in the ER diagram to properly handle many-to-many relationships, which the class diagram represented with simple association lines without needing to specify the implementation.

5. Appendix 2 - Data Dictionary

Entity: admin

Field Name	Data Type	Field Size	Description	Example
adminId	Integer	N/A	Admin's unique identifier	1
email	Varchar	255	Admin's email	christie@gmail.com
password	Varbinary	60	Admin's password	(hash) qiyh4XPJGsOZ2MEAYLkfWqeQ
twoFactorCode	Char	6	Code for two-factor authentication	345123
factorCodeExpiration	Datetime	N/A	Expiration date and time for 2FA code	2026-04-01 17:05:37

Constraints:

- *Primary Key: adminId (Autoincrement)*
- *NOT NULL: email, password, twoFactorCode, factorCodeExpiration*
- *UNIQUE: email*
- *CHECK: twoFactorCode matches pattern '^[0-9]{6}\$' (must be exactly 6 numeric digits)*

Entity: client

Field Name	Data Type	Field Size	Description	Example
clientId	Integer	N/A	Client's unique identifier	1
firstName	Varchar	255	Client's first name	Johnny
lastName	Varchar	255	Client's last name	White
email	Varchar	255	Client's email	johnny@gmail.com
phoneNumber	Varchar	20	Client's phone number	5142542836

Constraints:

- *Primary Key: clientId (Autoincrement)*
- *NOT NULL: firstName, LastName, email, phoneNumber*
- *UNIQUE: email*

Entity: ballroom

Field Name	Data Type	Field Size	Description	Example
ballroomId	Integer	N/A	Ballroom's unique identifier	1
roomName	Varchar	255	Ballroom's name	Princesse
minCapacity	Integer	N/A	Ballroom's minimum guest capacity	100
maxCapacity	Integer	N/A	Ballroom's maximum guest capacity	350
sizeSqFt	Integer	N/A	Ballroom's size in square feet	1350
roomPictures	Longblob		Gallery of photos of the ballroom	

Constraints:

- *Primary Key: ballroomId (Autoincrement)*
- *NOT NULL: roomName, minCapacity, maxCapacity, sizeSqFt, roomPictures*
- *UNIQUE: roomName*

Entity: foodCategory

Field Name	Data Type	Field Size	Description	Example
categoryId	Integer	N/A	Food category's unique identifier	1
categoryName	Varchar	255	Food category's name	Hot Fish

Constraints:

- *Primary Key: categoryId (Autoincrement)*
- *NOT NULL: categoryName*
- *UNIQUE: categoryName*

Entity: foodItem

Field Name	Data Type	Field Size	Description	Example
itemId	Integer	N/A	Food item's unique identifier	1
itemName	Varchar	255	Food item's name	Chicken Teriyaki
itemPrice	Decimal	10,2	Food item's price	5.99
extraPrice	Decimal	10,2	Food item's price if it's from another section	2.99
itemCategory	Integer	N/A	Food item's category	Vegetarian

Constraints:

- *Primary Key: itemId (Autoincrement)*
- *Foreign Key: itemCategory references foodCategory.categoryId*
- *NOT NULL: itemName, itemPrice, extraPrice, itemCategory*
- *UNIQUE: itemName*
- *DEFAULT: extraPrice = 0.00*

Entity: menu

Field Name	Data Type	Field Size	Description	Example
menuId	Integer	NNN	Menu's unique identifier	001
menuName	Varchar	100	Menu's name	Vegetarian
pricePerPerson	Decimal	10,2	Price per guest for the menu	15.99

Constraints:

- *Primary Key: menuId*
- *NOT NULL: menuName, pricePerPerson*
- *UNIQUE: menuName*
- *DEFAULT: pricePerPerson = 0.00*

Junction Table: menuFoodItem

Field Name	Data Type	Field Size	Description	Example
itemId	Integer	N/A	Food item's unique identifier	1
menuId	Integer	N/A	Food item's menu identifier	2

Constraints:

- *Primary key: menuId, itemId (composite)*
- *Foreign keys: menuId references menu.menuId, itemId references foodItem.itemId*
- *NOT NULL: menuId, itemId*

Entity: bar

Field Name	Data Type	Field Size	Description	Example
barId	Integer	N/A	Bar's unique identifier	1
barType	Varchar	50	Bar's type (e.g., free, paid, premium)	Premium
pricePerPerson	Decimal	10,2	Price per person for the bar	16.99

Constraints:

- *Primary Key: barId (Autoincrement)*
- *NOT NULL: barType, pricePerPerson*
- *DEFAULT: pricePerPerson = 0.00*
- *CHECK: barType IN 'Free', 'Paid', 'Premium'*

Entity: services

Field Name	Data Type	Field Size	Description	Example
serviceId	Integer	N/A	External service's unique identifier	1
serviceName	Varchar	50	External service's name	Souvlaki
serviceEmail	Varchar	255	External service's email	souv@gmail.com
servicePhoneNumber	Varchar	20	External service's phone number	514-556-6769

serviceDescription	Varchar	255	External service's description. What will they do?	Authentic Greek food
serviceType	Varchar	50	External service's type. What do they offer?	Catering

Constraints:

- *Primary Key: serviceId (Autoincrement)*
- *NOT NULL: serviceName, serviceEmail, servicePhoneNumber, serviceDescription, serviceType*

Entity: event

Field Name	Data Type	Field Size	Description	Example
eventId	Integer	N/A	Event's unique identifier	1
clientId	Integer	N/A	Client's unique identifier	2
ballroomId	Integer	N/A	Ballroom's unique identifier	3
menuId	Integer	N/A	Menu's unique identifier	4
barId	Integer	N/A	Bar's unique identifier	5
paymentId	Integer	N/A	Payment's unique identifier	6
barOpenTime	Time	N/A	The time at which the bar will start serving guests	20:00:00
barCloseTime	Time	N/A	The time at which the bar will stop serving guests	2:00:00
eventDate	Date	N/A	Date of the event	2026-08-08

eventTime	Time	N/A	Time of the event	17:00:00
guestQuantity	Integer	N/A	Number of guests attending	320
eventType	Varchar	50	Type of event (e.g.: Wedding, Birthday, etc.)	Wedding
eventDescription	Varchar	255	A description of the event	John and Christie's wedding
totalPrice	Decimal	10,2	Total price for the event	7999.99
depositAmount	Decimal	10,2	Amount required as a deposit	250.00
depositPaid	Decimal	10,2	Amount paid by client	500.00
paymentPlan	Varchar	50	Client's preferred payment plan (e.g., full, installments, etc.)	Installments
paymentMethod	Varchar	50	How the client chooses to pay	E-transfer
tableArrangement	Longblob		How the ballroom tables are arranged	
eventStatus	Varchar	50	Current status of the event (e.g., Cancelled, Declined, etc.)	Confirmed

Constraints:

- **Primary Key:** eventId (Autoincrement)
- **Foreign Keys:** clientId references client.clientId, ballroomId references ballroom.ballroomId, menuId references menu.menuId, barId references bar.barId, paymentId references payment.paymentId
- **NOT NULL:** clientId, ballroomId, paymentId, eventDate, eventTime, guestQuantity, eventType, eventDescription, totalPrice, depositAmount, depositPaid, paymentPlan, paymentMethod, tableArrangement, eventStatus
- **DEFAULT:** eventStatus = 'Pending'

- CHECK: paymentPlan IN 'Full', 'Installments'. paymentMethod IN 'Cash', 'E-transfer', 'Debit', 'Cheque'. eventStatus IN 'Confirmed', 'Pending', 'Done', 'Cancelled'
- NULLABLE: menuId, barId, barOpenTime, barCloseTime

Junction Table: eventService

Field Name	Data Type	Field Size	Description	Example
eventId	Integer	N/A	Event's unique identifier	1
serviceId	Integer	N/A	Service's unique identifier	2

Constraints:

- Primary key: eventId, serviceId (composite)
- Foreign keys: eventId references event.eventId, serviceId references services.serviceId
- NOT NULL: eventId, serviceId

Entity: payment

Field Name	Data Type	Field Size	Description	Example
paymentId	Integer	N/A	Payment's unique identifier	1
eventId	Integer	N/A	Event's unique identifier	2
amountPaid	Decimal	10,2	The amount the client has paid from the total price	4999.99
amountLeft	Decimal	10,2	The amount remaining from the total price to be paid	2999.99
paymentDate	Date	N/A	When the next payment is due	2020-05-15

Constraints:

- Primary Key: paymentId (Autoincrement)
- Foreign key: eventId references event.eventId
- NOT NULL: eventId, amountPaid, amountLeft, paymentDate
- UNIQUE: eventId

6. Appendix 3 - Query descriptions

Since we worked on this ERD for our eCommerce class, we based the first version mainly on the UML class diagram from deliverable 3. Then when it came to thinking about normalizing the database, we decided to add a payment table that tracks all payments that the clients must pay. We connected the *Payment* table with the *Client* table with a 1:many and 1:1 relationship since a client can make 1 or many payments if they choose to pay in installments.

We also thought it would be a good idea to put attributes from the Events table to Payment since there were some attributes that better suited the roles and responsibilities of the Payment table. We put attributes like “depositAmount”, “depositPaid”, “totalPrice”, and “paymentPlan” from Events to Payment simply because we mutually agreed that it would make more sense for a table that handles payments to have these attributes.

Another thing we took from Events and moved to another table was moving attributes like “tableArrangement” in the Ballroom table and moving attributes such as “barOpenTime” and “barCloseTime” to the Bar table. As always, the reason we decided to normalize it this way was to make sure certain attributes stay with the tables whose attributes and roles are similar.

7. Team Logbook

Meeting Number	Participants	What was discussed?	Where?	Work done	Duration (date and minutes)
1	Cyril, Daniel, Fairoz, Karim	- Went over who will do what for this deliverable	D-241	- ERD - Finishing up deliverable 4	Wednesday, April 1, 2026 75 minutes
2	Cyril, Daniel, Fairoz, Karim	- Discussed how the normalization processes will be divided among teammates	D-241	- Created a payment table - Cyril asked Alex whether our ERD looked normalized on Thursday, April 9, 2026	Wednesday, April 8, 2026 82 minutes
3	Cyril, Daniel, Karim	- Did 2 kahoots - Briefly discussed what appendix 3 was about	D-241	- Couldn't do anything since the wifi was blocked throughout the whole building	Wednesday, April 15, 2026 70 minutes